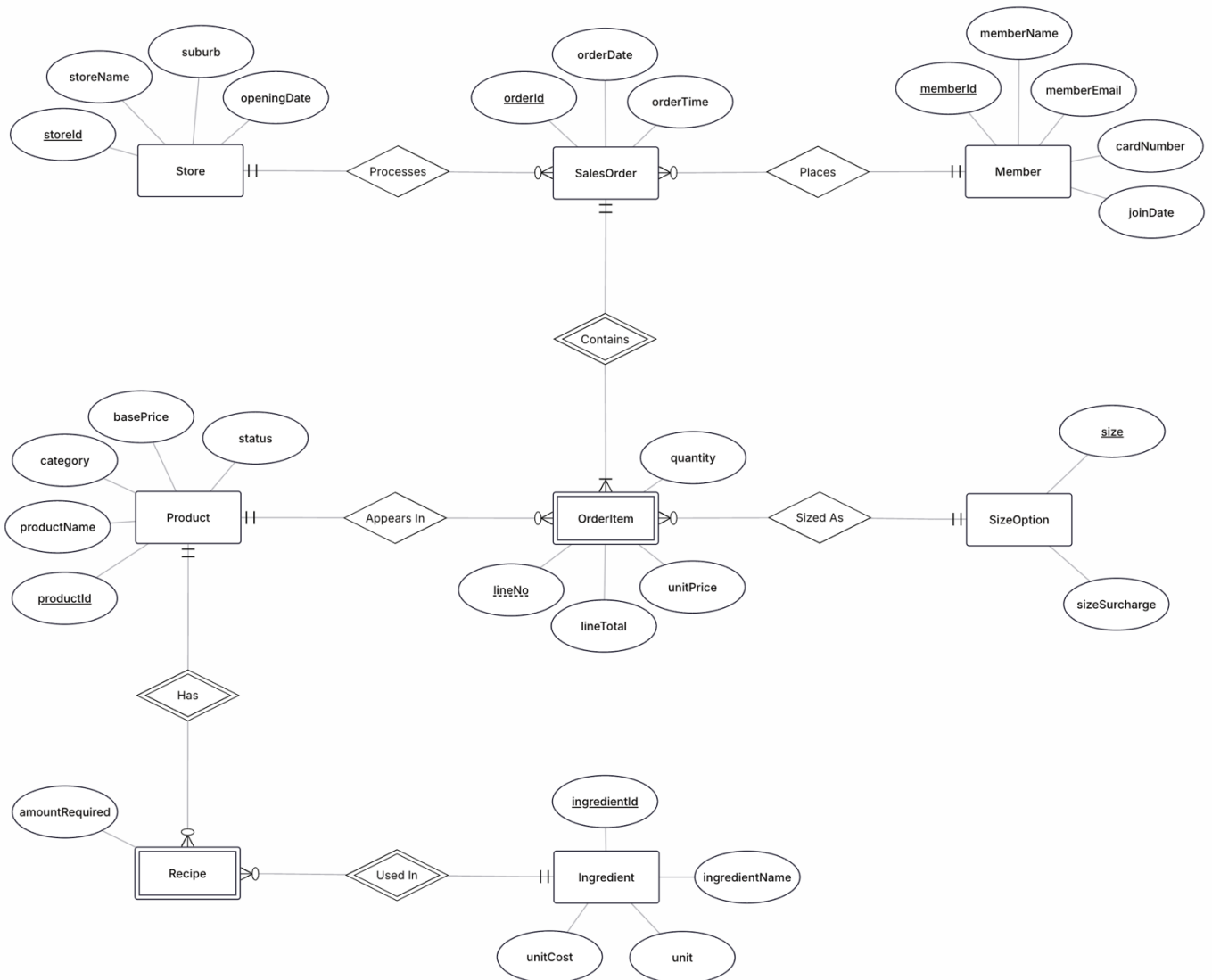


Entity-Relationship Diagram (ERD) of Bubble Trouble



Design Commentary

1. Why is OrderItem separate from SalesOrder?

General receipt details, such as order date, order time, store location, and member details, are stored in **SalesOrder**. By placing individual line items in **OrderItem**, duplicate order details are prevented across multiple rows. Redundancy is avoided through this setup, and many items can be attached to one transaction without breaking database normalization rules.

2. Why is the key of OrderItem naturally composite?

OrderItem is treated as a weak entity because full identification cannot be done without SalesOrder. Line numbers (lineNo) are restarted for every new order, which means item numbers like 1, 2, or 3 are repeated all over the database. Because lineNo is only a partial key, unique identification is achieved by joining orderId with lineNo. A composite key of (orderId, lineNo) is created to identify every line item across the system.

3. Why is Recipe required instead of storing ingredients directly in Product?

A many-to-many link is formed between products and ingredients, since several ingredients are needed for one drink and individual ingredients are reused across many drinks. Direct inclusion of ingredient lists inside Product is blocked by First Normal Form rules. The Recipe bridge table is added to resolve this link, allowing amountRequired to be saved as an attribute that tracks ingredient amounts for each drink.

4. Why is historical unitPrice stored in OrderItem even though a current price can be calculated from Product and SizeOption?

Base product prices and size surcharges stored in Product and SizeOption are updated over time. If item prices were calculated dynamically using current menu tables, past sales totals would be changed whenever prices were adjusted. By saving unitPrice directly in OrderItem at the time of purchase, historical transaction values are locked in for accurate reporting.